

HMI Copilot

From a PLC tag list and one sentence of English to a finished EcoStruxure project.

Target platform · EcoStruxure Operator Terminal Expert 4.4

A generated project opens in the product. It was never saved by the product.

Four costs, and they compound

Pain point	What it actually looks like	Where it hurts
Manual screen development	An engineer hand-places every lamp, bargraph and trend, then repeats it for 40 near-identical equipment screens.	Hours per screen, times the plant
Expert-driven configuration	Only senior engineers know which object, animation and colour standard applies. Juniors block waiting on them.	Seniors become a bottleneck
Complex tag integration	Hundreds of PLC symbols mapped by hand onto object properties. One wrong binding stays invisible until commissioning.	Silent until it is expensive
Inconsistency	Two engineers on one project produce two differently-named, differently-coloured HMIs.	Operators pay for it daily

The binding problem is the sharp one. A wrong binding is not a drawing error — it is pump 2's flow displayed under pump 1's label, discovered during an upset.

The question we asked before building anything

Does the output actually load into the engineering tool?

Everything else is decoration. A beautiful generated screen that EcoStruxure refuses to open is a mockup, and a judge who opens the file finds out in ten seconds.

So the first thing we did was take apart the files the product itself ships — Blank.eote, two demo projects and three samples — and find out whether a project could be written from scratch at all.

It could. That answer is the foundation of everything that follows.

The project format is fully legible

```
<project>.eote          ZIP   (nested entries use BACKSLASH separators)
|-- Project.dat, Target.dat  JSON   project identity, panel model, resolution
|-- Variables.db           SQLite the tags
|-- Alarm.db              SQLite alarm groups and alarms
|-- Bindings.dat          JSON   Sources[] -> Bindings[] -> Targets[] graph
+-- Screens\<guid>\Screen.dat JSON   the object tree
```

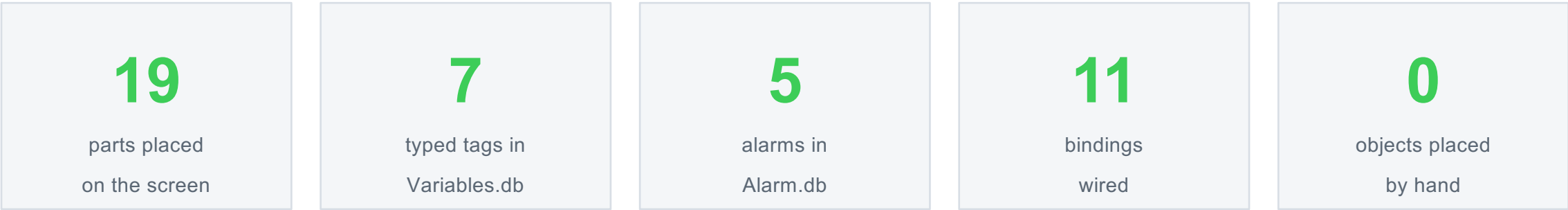
No proprietary binary. No signature. No encryption. Every project carries `_metadata` with "EncryptionInfo": null and "SignatureInfo": null.

Four properties make automation possible: everything meaningful is plain JSON or SQLite; nothing is signed or encrypted; bindings are declarative; and the geometry — absolute Location, Width and Height inside a ViewBox — maps one-to-one onto SVG.

That last one carries the whole user experience. It is what lets one model drive both the preview and the file.

Built, not proposed

Two .eote files were written from scratch and open in EcoStruxure Operator Terminal Expert 4.4. Neither was ever saved by the product.



File	Written by	Status
HMICopilot_PumpStation.eote	tools/make_project.py — the Python reference	opens in OTE 4.4
HMICopilot_TS.eote	web/src/lib/ote/packager.ts — the browser path	opens in OTE 4.4

Two independent implementations — Python and TypeScript — agreeing on the same output is a far stronger claim than either passing its own tests.

One model, two renderers — the preview cannot lie

The same JSON that becomes the project file drives the live preview. There is no second model of the screen, no “preview approximation” that drifts from the export.



What the engineer approves is what OTE opens.

Most tools in this space render a pretty approximation and generate the real artifact separately. The two drift, and the customer finds the drift. Here they are the same object, so drift is not unlikely — it is impossible.

The one rule, enforced by the compiler

The canvas may only render what the packager can emit.

A rule written in a document is a rule someone breaks at 2am. So this one is a type error instead.

ScreenRenderer switches exhaustively over the part union in schema.ts, with a never assignment on the default branch. Add a part to the schema and the canvas stops compiling until it can draw it.

The other half is a test: canvas.test.ts restates the Python reference renderer's rules independently, so the two renderers cannot drift together and still pass.

The visible corollary: the theme toggle themes the application and never the HMI screen, whose colours are palette indices from the project's own colour set.

Grounded twice, so invented properties are impossible

Grounding	Source	What it prevents
Machine-readable schemas	Buildtime/PropertyDefinitions/ — the product’s own definitions of Screen, Variable, 25 parts and 13 layout objects	A model inventing a property that does not exist. Structurally impossible, not merely unlikely.
Real part shapes	50 part types extracted from the shipped sample projects into reference/part_examples.json	A model guessing at the shape of a Lamp. Every shape we emit was observed in a project the product wrote.
zod at the boundary	web/src/lib/ote/schema.ts — a discriminated union parsed at runtime	A malformed object reaching the canvas or the packager at all.

The model is never asked “what should a pump screen look like?” in the abstract. It chooses among real parts, with real properties, from the product’s own vocabulary — which is why the output is boring in exactly the way engineering output should be boring.

We meet the product at the artifact boundary

```
HMI Copilot          BOUNDARY          EcoStruxure OTE
(Next.js / TypeScript) (files - language-agnostic) (C# / C++ / Qt / Lua)
|                   |                   |
generated JSON --> project.eote -+--> File > Open Project --> a built HMI
```

EcoStruxure Buildtime is a .NET/C# shell with native Qt5/C++ modules. RunTime is C++/Qt5 with embedded Lua. We do not merge into any of it.

That is judgement, not a workaround. Touching the internals would need an SDK we do not have and code signing, would break on every release, and would amount to asking Schneider to maintain a fork of its own product.

Staying at the file boundary means the tool works with any OTE 4.4+ installation, survives upgrades, and could actually ship. And both layers already ship Qt5WebEngine, so this same interface can be docked inside the tool later with no new technology decision.

Eight steps from a tag export to a file OTE opens

Step	Stage	In	Out
1	Tag Ingestion	.csv / .txt / .xlsx export	Normalised {name, dataType, address, scanRate, comment}
2	Equipment Inference	Tag names and comments	Clusters: PMP_101_RUN + _FLT + _SPD = one pump
3	Part Selector	Each tag and its role	The right part from the 50 the product ships
4	Layout Planner	Parts and panel size	Positions inside a VBox, reading order respected
5	Screen Generator	Layout + schema	Screen.dat JSON constrained by the .propDef schema
6	Binding Resolver	Tags and placed objects	The Sources / Targets / Bindings graph
7	Validation Engine	The whole project	Findings, each clickable back to its object
8	Project Packager	Everything above + skeleton	.eote — screens, Variables.db, Alarm.db, Bindings.dat

Step 2 is where expert knowledge lives: a pump should have a fault alarm, so a missing one is flagged rather than silently skipped.

Stack, and why each piece

Layer	Choice	Why
Framework	Next.js (App Router), full stack	One repo, one language, one deploy. No CORS, no second dev server.
Canvas	Inline SVG, one node per object	Direct mapping to the OTE screen model. Click-to-select maps straight back to the JSON node.
State	Zustand + immer	A JSON tree with cheap undo/redo and snapshots.
Transport	SSE from a route handler	Streams one complete object at a time — never a partially-parsed JSON tree.
Model	Gemini, Claude as the alternative	Schema-grounded structured output. One model call per generation.
Packager	jszip + sql.js, Node runtime	No native module, so it runs anywhere including serverless.

Three packager constraints learned the hard way: the route must declare nodejs runtime not Edge; ZIP entry names must use backslash separators; only Variables.db and Alarm.db are written, the rest pass through verbatim.

01

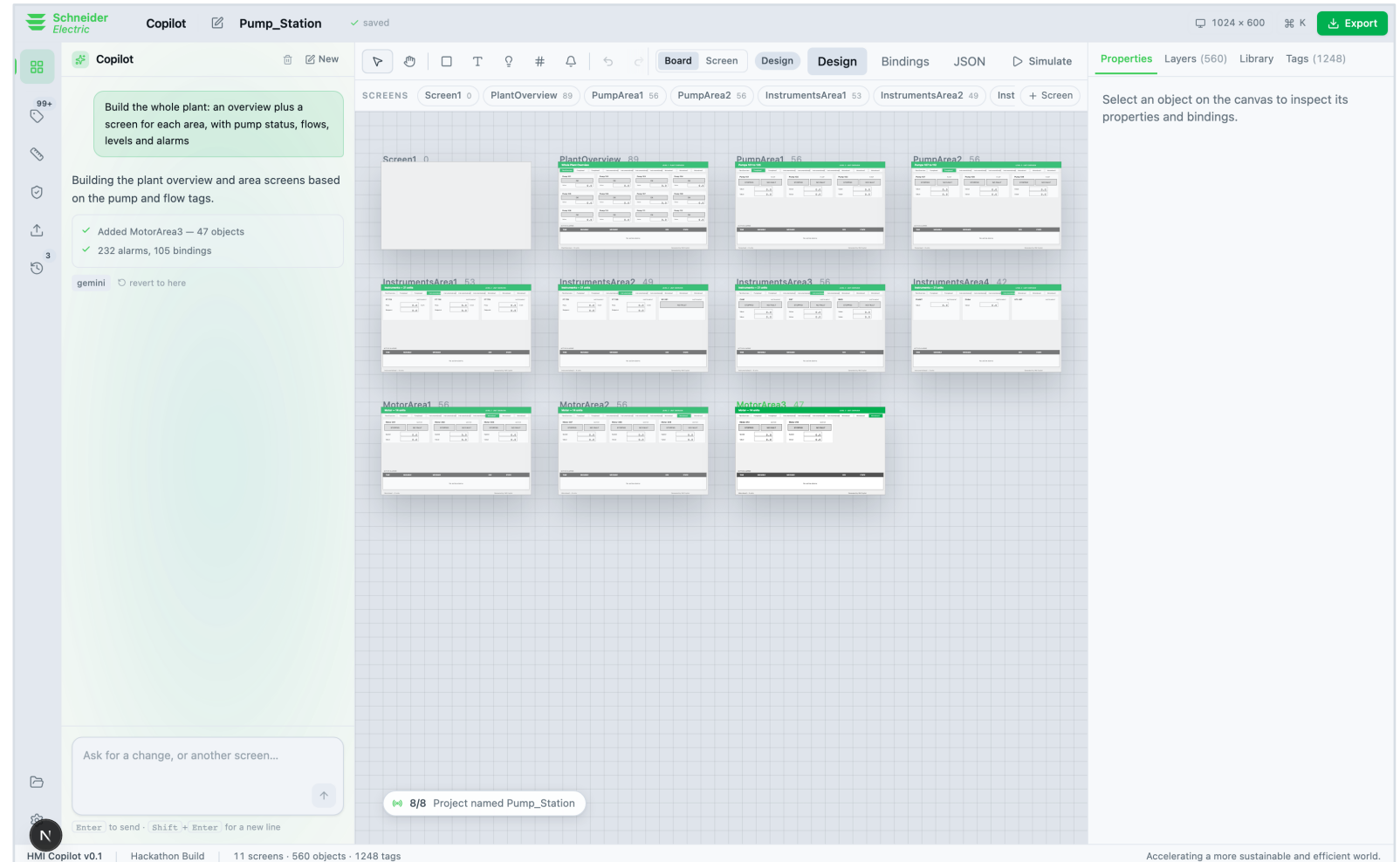
What we built

The product, in seven screens.

The board — every screen in the project at once

Eleven screens generated from one sentence, laid out as a board you can zoom and pan.

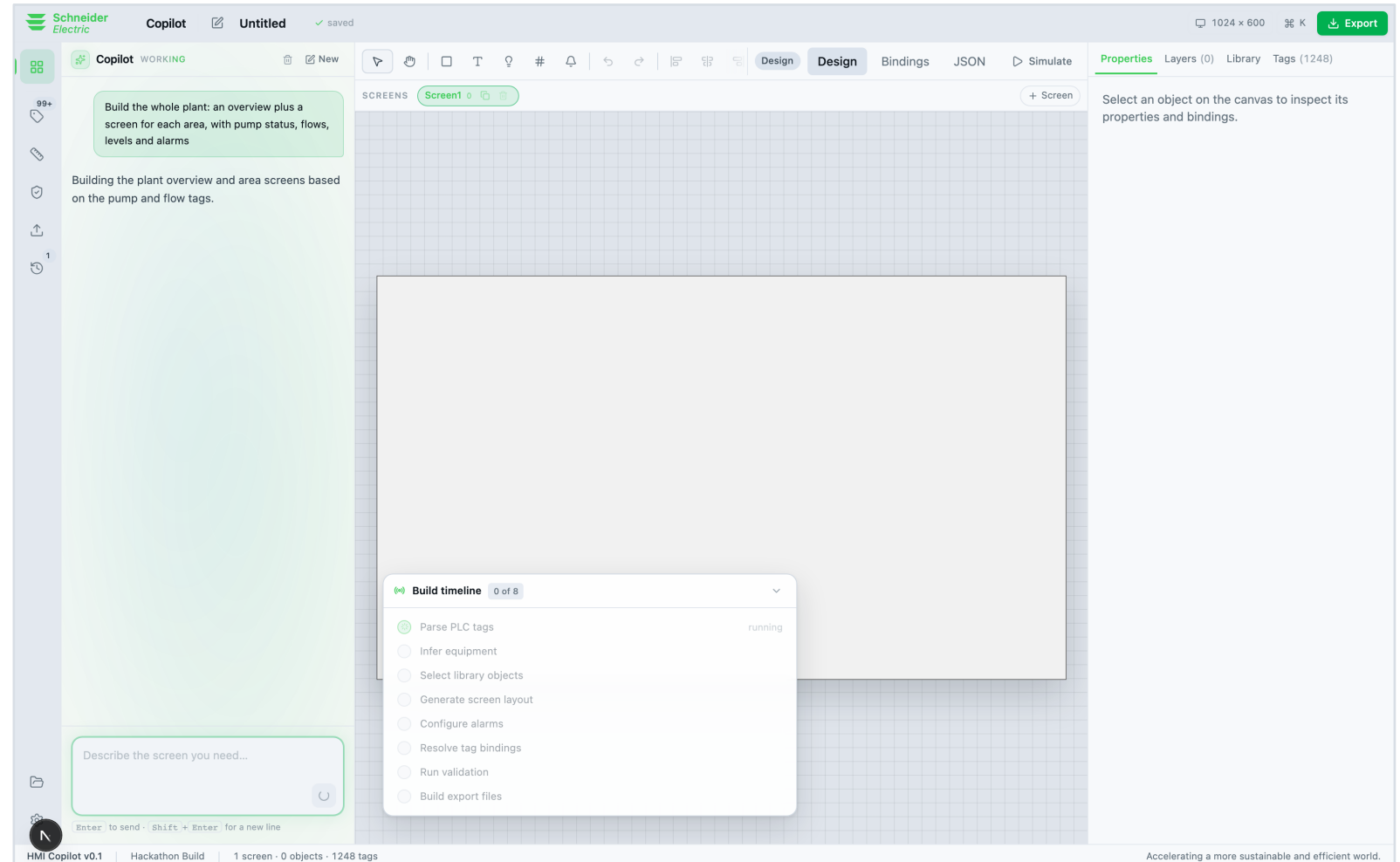
A 1,248-tag plant is not one screen; it is a screen hierarchy. The board is where you see the hierarchy, and the strip above it is where you move between the screens in it.



The run overlay — the AI's work in engineering language

Eight steps, each reporting what it produced, each clickable to select the objects it created.

Not “Thinking...”. Parse PLC tags → 1,248 tags. Infer equipment → pumps, instruments, motors. Configure alarms → 232. When the run finishes the overlay settles into a single line and gets out of the way.



The inspector — derived from the schema, not hand-written

Select any object and edit its real properties. The fields are generated by walking the zod union at runtime.

A field is a colour field because its shape is `{Color:{Value}}`, not because someone named it “Fill”. Add a property to the schema and the inspector grows it for free. Layers, Library and Tags sit beside it as tabs.

The screenshot displays the Schneider Electric HMI Copilot interface. The main workspace shows a design for 'Pump Station 1' with a green header. The design includes a 'PUMPS' section with 'PUMP 1 STOPPED' and 'PUMP 2 STOPPED' buttons, and a 'PROCESS' section with 'Flow' and 'Level' readouts. Below these is an 'ACTIVE ALARMS' table with columns for TIME, VARIABLE, MESSAGE, SEV, and STATE. The interface also features a left sidebar with a 'Copilot' chat window, a top toolbar with various design tools, and a right sidebar with a 'Properties' panel for the selected 'Lamp' object.

Properties Panel (Lamp #Lamp_PUMP1_RUN):

- Geometry:** X: 40, Y: 120, W: 200, H: 76
- Data binding:** PMP_101_RUN drives CurrentValue
- General:**
 - Type: Lamp
 - Name: Lamp_PUMP1_RUN
 - Unique id: dabbce5b1-b809-4576-b4bb-15d78108
 - Location: 40, 120
 - Width: 200
 - Height: 76
- States:**
 - Off:
 - Text: PUMP 1STOPPED
 - Text color: 1 · #030303
 - Font: 13, Bold, Italic
 - Text layout: Middle
 - Wrap: On

Footer: HMI Copilot v0.1 | Hackathon Build | 1 screen · 19 objects · 7 tags | Accelerating a more sustainable and efficient world.

The binding map — the stated pain point, made visible

Tags on one side, object properties on the other, drawn from the real Sources / Targets graph the .eote contains.

This is the highest-value view in the product, because tag integration is the pain point the brief names, and today it is invisible until commissioning.

The screenshot shows the Schneider Electric HMI Copilot interface for a project named 'Pump_Station_Demo'. The 'Bindings' tab is active, displaying a list of PLC tags on the left and object properties on the right. The tags include PMP_101_RUN, PMP_101_FLT, PMP_102_RUN, PMP_102_FLT, FT_101_PV, LT_101_PV, and LT_101_HI. The object properties include Lamp_PUMP1_RUN.CurrentValue, Lamp_PUMP1_FLT.CurrentValue, Lamp_PUMP2_RUN.CurrentValue, Lamp_PUMP2_FLT.CurrentValue, Num_Flow.CurrentValue, Num_Level.CurrentValue, and AlarmGroup1.Alarm1.HiHi.VariableName through AlarmGroup1.Alarm5.HiHi.VariableName. The bindings are visualized as lines connecting the tags to the object properties. A legend at the bottom explains the binding types: green for display binding (tag drives an object property), orange for alarm binding (tag is the alarm trigger), and red for unbound (the object displays a value nothing drives).

The right sidebar shows the properties of the selected 'Lamp' object, including geometry (X: 40, Y: 120, W: 200, H: 76) and data binding (PMP_101_RUN drives CurrentValue). The 'General' section shows the object type as 'Lamp', name as 'Lamp_PUMP1_RUN', and unique ID as 'dabbc5b1-b809-4576-b4bb-15d78108'. The 'States' section shows the 'Off' state with text 'PUMP 1STOPPED', text color '#030303', font size '13', and text layout 'Middle'.

At the bottom, the status bar indicates 'HMI Copilot v0.1 | Hackathon Build | 1 screen - 19 objects - 7 tags' and 'Accelerating a more sustainable and efficient world.'

Live simulation — behaviour before hardware

Flip to Live and a process model drives the canvas: lamps change state, numerics move, alarms raise and clear.

Numerics are driven through their own alarm setpoints, so every alarm the project defines actually fires. Lead and standby pumps behave differently. This is the demo moment.

The screenshot displays the Schneider Electric HMI Copilot interface for a project named 'Pump_Station_Demo'. The interface is divided into three main sections:

- Left Sidebar (Copilot):** Contains a chat window with instructions like 'Describe what you need and keep going until it is right.' and a list of suggested actions such as 'Create a screen for <equipment>, showing status and readings'.
- Central Canvas:** Shows a live simulation of 'Pump Station 1'. It includes a 'PUMPS' section with 'PUMP 1 RUNNING' (green) and 'PUMP 2 STOPPED' (grey). A 'PROCESS' section displays 'Flow' at 57.1 LPM and 'Level' at 102.4 %. Below this is an 'ACTIVE ALARMS' table with the following data:

TIME	VARIABLE	MESSAGE	SEV	STATE
05:34:39	PMP_102_FLT	Pump 2 fault	5	ACTIVE
05:34:39	LT_101_PV	Tank level critically high	5	ACTIVE
05:34:39	LT_101_HI	Tank high level	3	ACTIVE
05:34:39	LT_101_PV	Tank level high	3	ACTIVE
- Right Sidebar (Properties):** Shows the properties for a selected 'Lamp' object (#Lamp_PUMP1_RUN). It includes 'Geometry' (X: 40, Y: 120, W: 200, H: 76), 'Data binding' (PMP_101_RUN drives CurrentValue), and 'General' settings (Type: Lamp, Name: Lamp_PUMP1_RUN, Unique id: dabbce5b1-b809-4576-b4bb-15d78108).

The bottom status bar indicates 'HMI Copilot v0.1 | Hackathon Build | 1 screen - 19 objects - 7 tags' and 'Accelerating a more sustainable and efficient world.'

Tag import — corrections reported, never applied silently

1,248 tags parsed, five names corrected, every correction listed with its before and after.

OTE silently drops names that break its rules on import. Here they are caught, auto-corrected, and the change is shown. This is the single most trust-relevant behaviour in the product.

Schneider Electric

Copilot

Untitled

saved

1024 x 600

K

Export

Import PLC Tags

Import tags from your PLC export file.

Drag and drop your file here

or click to browse — .csv · .txt · .xlsx

Plant_Tags.csv

1,248 tags detected

Parsed

5 names corrected

Tag Summary

Total Tags

1,248

BOOL

921

INT

21

DINT

13

REAL

281

LREAL

6

STRING

6

Parsed tags

1,248

All types

PLC Tag	Type	Address	Comment
PMP_101_RUN	BOOL	%MX500.2	Pump 1 running
PMP_101_FLT	BOOL	%MX500.4	Pump 1 fault
PMP_101_AVAIL	BOOL	%MX500.6	Pump 1 available
PMP_101_START	BOOL	%MX501.0	Pump 1 start command
PMP_101_STOP	BOOL	%MX501.2	Pump 1 stop command
PMP_101_HOURS	DINT	%MD4012	Pump 1 run hours
PMP_101_AMPS	REAL	%MD4014	Pump 1 motor current
PMP_101_SPEED	REAL	%MD4016	Pump 1 speed feedback
PMP_102_RUN	BOOL	%MX502.2	Pump 2 running
PMP_102_FLT	BOOL	%MX502.4	Pump 2 fault
PMP_102_AVAIL	BOOL	%MX502.6	Pump 2 available

HMI Copilot v0.1

Hackathon Build

1 screen · 0 objects · 1248 tags

Accelerating a more sustainable and efficient world.

Validation, then export — caught at the desk, signed off on paper

Seven rule families run before export: naming, binding integrity, type mismatches, completeness, alarms, unused tags, standards.

Every finding carries the objectId of the thing that caused it, so the binding map and the validation list can never disagree. Export produces the .eote, a Variables.csv, and a standalone HTML sign-off report with no stylesheet, font host or script — because a commissioning laptop has no internet.

The screenshot shows the Schneider Electric Copilot interface for a project named 'Pump_Station_Demo'. The 'Generate & Export' section is active, showing a summary of project data and a validation error.

What will be written		
Project	Target	Screens
Pump_Station_Demo	HMIGTO6310 · 1024 x 768	1
Objects	Variables	Alarms
19	7	5
Bindings		
11		

Validation Re-run

1 error — the file will still be written, but fix these before the project goes to a panel.

Files to generate

- ☒ **EcoStruxure project (.eote)**
Screens, variables, alarms and the binding graph, packaged the way the product writes them.
- ☒ **Variables (.csv)**
The tag list, in the columns our own importer reads back.
- ☒ **Validation report (.html)**
Every finding with the object that caused it. Opens with no internet, so it can be read on a commissioning laptop.

Generate files

Export preview
Nothing generated yet. Choose what you need and press Generate files.

Footer: HMI Copilot v0.1 | Hackathon Build | 1 screen · 19 objects · 7 tags | 1 errors | Accelerating a more sustainable and efficient world.

THE RESULT

One sentence. A whole plant.

“Build the whole plant: an overview plus a screen for each area, with pump status, flows, levels and alarms”

11

screens

560

objects

232

alarms
configured

1,248

tags
ingested

0

errors
0 warnings

From the 1,248-tag water treatment export, with a navigation model between the screens — a plant overview, then a screen per area, and the buttons that move between them.

One measured run. Screens and objects vary with the model — other runs produced 8 / 434 and 10 / 486. What did not vary on any run: 232 alarms, 1,248 tags, 5 corrections, every binding resolved, 0 errors.

Hand-placing several hundred objects and making every binding is the work this replaces. Apply your own hourly rate.

Two workstreams that could never touch the same file

	FORMAT	SURFACE
Owns	Everything touching the .eote file format	Everything the engineer sees
Files	lib/ote, lib/ai, lib/tags, validation, app/api, scripts, fixtures, tools	components, app routes, store, lib/sim, styles
Delivered	The packager, tag ingestion, inference, the generation pipeline, validation rules, the sign-off report, the symbol index	The design system, the canvas, the inspector, the binding map, simulation, the supporting screens, the demo path
Machine	Windows, with a licensed EcoStruxure 4.4 installation	macOS

Three files were frozen without a conversation: schema.ts, events.ts and package.json — the contracts between the two halves. The interesting constraint: FORMAT had the licensed install and SURFACE did not, so the format work had to be provable on a machine that could not run the product.

Testing, and the engineering by the numbers

467

tests in the suite

454

run anywhere

13

need a licensed
EcoStruxure install

0

failing

60

commits

18,251

lines of TypeScript

474

shipped symbols indexed

50

real part types extracted

Two implementations, structurally diffed

packager.test.ts diffs the TypeScript packager against the Python reference — same entries, same binding graph, same rows. Two independent implementations agreeing is stronger than either passing its own tests.

The renderer checked against a restatement, not an import

canvas.test.ts restates the Python renderer's rules independently, so the two cannot drift together and still pass.

WHAT WENT WRONG

The issue log

What broke	Why	How it was fixed
The ZIP would not open in OTE	Nested entry names use backslash separators, which every ZIP library normalises to forward slashes.	Write the entry names the product itself writes, and guard the separator with a test.
Symbols lost their geometry	The graphics indexer dropped Commands and Points, keeping only derived path data — so a symbol could not round-trip.	Keep all three, and assert with a test that they can never drift.
Every large generation died	A React update loop: effect dependencies that changed identity on every store write. 232 alarm events blew past the nested-update limit.	Dependencies narrowed to one, and a referentially idempotent idle state.
Half the toolbar was unreachable	719px of a 1,498px toolbar sat past the right edge on a 1440px laptop, including Simulate.	The run-state group pinned outside the scroll region.
The Copilot column went invisible	Three new ground tokens were added to the dark theme and never given light-theme values, so dark grounds met dark ink.	Contrast ratio 1.03, measured in the browser. Light values added.

Two lessons that changed how we work

Parsed 1248 tags • Placed 29 objects • Generation failed: Maximum update depth exceeded

1 • A demo fixture small enough to be convenient is small enough to hide the bug that matters.

The seven-tag fixture never triggered it. A 1,248-tag export did, every time. Every sample we ship now exercises a different path through the importer rather than flattering it.

2 • A passing test suite does not tell you whether a button is reachable.

The crash was found by a person clicking. So we started driving the real product in a real browser at 1280, 1440 and 1920 — measuring overflow, clipped text, hit-target size and colour contrast. That found the off-screen Simulate button, a timeline that said “0 of 8” while showing seven, two hydration mismatches, and the invisible Copilot column.

Zero console errors across all eleven routes, at every width, in both themes.

The decision that comes before the architecture

Every .eote we write contains Schneider’s own database files, copied through verbatim — Recipe.db, Security.db, Language.db, DriverConfig.db. Each developer extracts their own copy from their own licensed installation.

Right for a hackathon. Not a posture a product can ship in: a hosted service needs the skeleton on the server, and a server holding those files is redistributing them.

	How it works	Verdict
A. Local generation	Runs on a machine that already has a licensed OTE install. The skeleton never moves.	Build this
B. Server plans, browser packages	Layout server-side; the .eote assembled in the browser from a skeleton on the engineer’s own disk.	Don’t — Chromium-only, fragile
C. Schneider blesses it	An OEM agreement covering a redistributable skeleton, or better, an official project API.	Pitch this

Build for A, pitch for C — they are compatible. A is also the honest fit: plant engineering happens on restricted, frequently air-gapped networks, and a cloud tool that phones a model API is a procurement fight at every site.

Four gaps, named plainly — and the safety position

#	Gap	What we do about it
1	It can only write, never read. Most HMI work is brownfield — adding a pump to a station built in 2019, not starting from nothing.	Build the reader as a preservation problem first: model what you understand, carry the rest through untouched, assert a byte-identical round trip.
2	Persistence is local only. Autosave to localStorage survives a reload; it does not survive a new machine.	For architecture A this is a local database and a file on disk — not a multi-tenant backend.
3	Nothing talks to a PLC. Generated variables are internal, so the export is a drawing, not a project.	Device driver configuration, written into DriverConfig.db rather than passed through.
4	Generation quality is unmeasured. No corpus, no scoring, no regression gate.	A golden corpus of 20–50 real tag exports with the screens an experienced engineer would draw. Ship nothing that regresses binding accuracy.

And the safety position: never auto-deploy. The engineer watches every object appear and approves before export. That is not a hackathon limitation to grow out of — it is the product’s licence to exist.

How it merges with the product — the integration ladder

Tier	Mechanism	Status
0 — Artifact	Generate the .eote; the engineer opens it. Zero coupling, no licence dependency.	Working today
1 — Lua scripting	Both layers embed Lua — the product's own sanctioned extension point.	Natural next step
2 — Managed plugin	Buildtime is .NET, so the generator can be repackaged as an in-process C# library.	Needs Schneider's SDK
3 — UI automation	Driving the OTE window programmatically.	Brittle. Not worth it

And it can live inside OTE later, with no new technology decision.

Both Buildtime and RunTime already ship Qt5WebEngine — an embedded Chromium — and the parts library already includes a WebBrowser object. The same interface can be docked inside the engineering tool, and surfaced on the panel at runtime for operator diagnostics, on a runtime Schneider already distributes.

02

The business model

Who pays, why, how much, and what stops someone else doing it.

Three buyers, one wedge

Buyer	What they feel	Why they buy	Priority
System integrators & panel builders	Fixed-price projects, margin pressure, HMI engineering is a large share of the hours.	Time saved converts directly into margin on work they have already quoted. They buy tools with their own money and decide fast.	THE WEDGE
End users — plants, utilities, OEMs	Inconsistency across contractors, standards drift, errors found at commissioning.	They own the house style. Standards packs make every contractor produce their HMI, not the contractor's.	Expand
Schneider Electric	OTE competes with Siemens WinCC and Rockwell FactoryTalk on engineering productivity.	A generative front end that runs on the format they already ship, with no change to the product.	Scale

Start with integrators: sharpest pain, shortest sales cycle, and they already have a licensed OTE installation on every engineering laptop — exactly what architecture A requires.

Price, and why a seat pays for itself

Tier	Who	Includes	Price
Individual	One engineer, one laptop	Local generation, all part types, symbol library, validation, export, sign-off report	€149 / engineer / month
Team	5–25 seats at an integrator	Plus shared standards packs, project templates, priority support	€119 / seat / month
Site / Enterprise	A plant or large integrator	On-prem or air-gapped install, bring-your-own or local model, audit trail, custom standards packs, conformance, SLA	from €40k / year
OEM	Schneider Electric	Bundled with or sold alongside OTE; revenue share or per-licence royalty	Negotiated

BREAK-EVEN (assumptions stated, customer changes them)

Fully-loaded automation engineer	EUR 70-100 / hour
Team seat	EUR 1,428 / year
Hours it must give back	$1,428 / 85 = \sim 17 \text{ hours} = \text{two working days}$

Break-even at two days a year is an easy question to answer honestly — and it does not depend on a savings percentage we have not measured. Illustrative pricing.

Go to market, and what stops someone copying it

	Land	Expand	Scale
Who	System integrators, 5–50 engineers	Their end customers — plants and utilities	Schneider’s channel
Motion	Direct, self-serve trial on a laptop that already has OTE	Standards packs authored to the customer’s house style	OEM agreement, bundled with OTE
Proof needed	A generated project opens. Already true.	Round-trip safety on brownfield, and the audit trail	Version-matrix conformance across OTE releases

Format knowledge that had to be earned

Fifty real part types, the machine-readable property definitions, the palette out of the product’s own Colors.lua, the naming rules, and the backslash separator that decides whether a file opens. None of it documented publicly; all verified by a file that opens.

Two implementations that agree, and a compiler-enforced rule

Anyone can write one packager; proving it correct is the work. A competitor who builds a preview separately from the generator ships drift their customers find.

Standards packs become switching cost

Once a customer’s conventions live in the tool as data and every contractor generates against them, leaving means renegotiating consistency across a supply chain.

Risks, and what we would build next

Risk	Severity	Mitigation
Schneider declines a redistributable skeleton	High	Architecture A needs no redistribution — the tool runs where a licence already is. The business survives a “no”.
A generated binding is wrong and nobody catches it	Critical	Never auto-deploy. The engineer approves before export, findings carry object ids, and the golden corpus gates every change to binding accuracy.
Format drift across OTE versions	Medium	Part schemas read from the installation at run time; the packager should refuse to write a version it has not been proven against.
Air-gapped sites cannot reach a model	Medium	A local model is a requirement, not a nice-to-have. The provider seam is one module today — cheap now, expensive once call sites spread.

Next, in order: take the licensing question to Schneider · start the .eote reader as a preservation problem · stand up the golden corpus. Then durable persistence and the audit trail. Then ISA-101 and ISA-18.2 rules and the version matrix. Then desktop packaging and a local model.

Deliberately cut: multi-tenancy, SSO, a marketplace, real-time collaboration. Two engineers do not edit one HMI screen at once.

IN SUMMARY

Where this stands

11

screens from
one sentence

560

objects, none
placed by hand

1,248

tags
ingested

467

tests

4.4

opens in OTE

Screens and objects are one measured run and vary with the model; alarms, tags, corrections and the zero-error result did not vary on any run.

The loop works, end to end

Tag export in, conversation, live canvas, validation, and a .eote that opens in EcoStruxure Operator Terminal Expert 4.4 — a file the product never saved.

The format layer is genuinely defensible

Two independent implementations, structurally diffed. The canvas cannot draw what the packager cannot emit, and the compiler enforces it.

What stands between this and a real engineer is not features

It is the licensing question, a reader that never loses what it does not understand, and a way to prove the generation is right. Everything else is sequencing.

The AI does not replace the HMI engineer.

It encodes the senior engineer's judgement — which part to use, which alarm every pump needs, which naming standard applies — and makes it available to everyone on the team, every time.

HMI Copilot · From Intent to HMI — Faster. Smarter. Safer.